

13.4

Autres problèmes

NSI TERMINALE - JB DUTHOIT

13.4.1 Découpe d'une barre

● Exercice 13.17

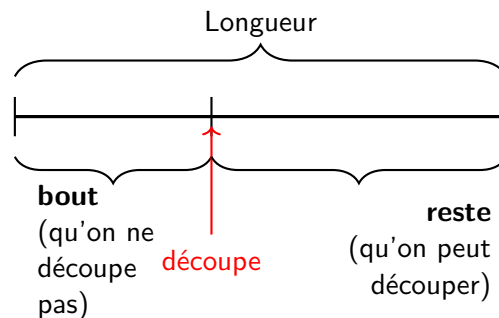
On dispose d'une barre de longueur donnée que l'on peut revendre d'un seul tenant ou en morceaux.

☛ L'objectif étant de maximiser le gain.

Par exemple, on considère une barre de longueur 8 et le tableau ci-dessous donne les prix en fonction de la longueur du morceau :

Longueur du morceau	0	1	2	3	4	5	6	7	8
Prix du morceau	0	2	3	8	10	13	15	16	21

1. On veut ici préparer le travail pour comprendre l'algorithme qui va suivre.



Pour une barre de longueur 1, on a :

decoupe	Prix bout	Reste (Gain max)	Prix
1	$P(1) = 2$	Taille 0 (= 0)	2

Pour une barre de longueur 2, on a :

decoupe	Prix bout	Reste (Gain max)	Prix
1	$P(1) = 2$	Taille 1 (= 2)	4
2	$P(2) = 3$	Taille 0 (= 0)	3

Pour une barre de longueur 3, on a :

decoupe	Prix bout	Reste (Gain max)	Prix
1	$P(1) = 2$	Taille 2 (= 4)	6
2	$P(2) = 3$	Taille 1 (= 2)	5
3	$P(3) = 8$	Taille 0 (= 0)	8

- a) Combien y a t'il de découpes possibles pour une barre de longueur 4? Quels sont les gains associés? (refaire le même type de tableau)
- b) Combien y a t'il de découpes possibles pour une barre de longueur 5? Quels sont les gains associés? (refaire le même type de tableau)

2. Voici un algorithme - Approche naïve - qui repose sur le principe suivant :

- Envisager toutes les longueurs possibles pour le premier morceau coupé.
- Calculer le gain de ce scénario : Prix du premier morceau + Gain maximum du reste de la barre.
- Comparer tous ces scénarios et conserver le plus rentable.

```

1 Fonction gain_max(taille, prix)
2   Données : taille (entier), prix (tableau de prix qui commence par zero)
3   Variables locales : meilleur, decoupe (entiers)
4   SI taille <= 0 ALORS
5     | renvoyer 0
6   meilleur ← 0
7   decoupe ← 1
8   TANT QUE decoupe <= taille FAIRE
9     | meilleur ← max(meilleur, prix[decoupe] + gain_max(taille - decoupe, prix))
10    | decoupe ← decoupe + 1
11  renvoyer meilleur
12 fin

```

Implémenter cet algorithme en python.

On devrait trouver :

```

>>> P = [0, 2, 3, 8, 10, 13, 15, 16, 21]
>>> gain_max(4, P)
10
>>> gain_max(7, P)
18

```

3. Tester de nouveau cet algorithme avec le tableau de prix suivant :

```

# Tableau des prix (on ajoute 0 au debut pour que l'indice corresponde a la ta
P = [0, 2, 3, 8, 10, 13, 15, 16, 21]
P\_long = P + [22] * 35 # on a donc des prix pour une longueur de 43

```

⚠ La complexité de l'algorithme étant exponentielle, nous allons essayer d'améliorer l'algorithme.

4. Proposer un programme python Top down, et le tester avec la longueur de 43.
5. Proposer un programme python Bottom up, et le tester avec la longueur de 43.
6. Essayer d'améliorer l'algorithme précédent pour qu'il donne la découpe optimale.

13.4.2 Pyramide de nombres

Exercice 13.18

Une pyramide de nombre est un graphe donc les sommets sont des nombres. Chaque sommet d'un même niveau a deux arrêtes vers le bas. Deux sommets voisins d'un même niveau sont reliés à un même sommet du niveau suivant.

```

      3
     2 4
    2 7 5
   9 5 4 6

```

Ici on peut suivre le chemin 3 - 4 - 7 - 5 mais pas 3 - 2 - 5 : les sommets 2 et 5 ne sont pas reliés.
 Objectif : Déterminer la valeur maximale de la somme des chemins traversant une pyramide de nombre.

1. Quel est le chemin qui donne la somme maximale dans l'exemple ci-dessus ? ***
2. Choix d'une structure de données adaptée pour les pyramides :
 $T[i, j]$ est la valeur du nombre situé à l'étage i en partant du bas et en position j en partant de la gauche.
 • Toujours sur l'exemple, $T[1,1] = 9$, $T[1,2] = 5$, $T[2,1] = 2$, $T[3,1] = 2$. Donner les valeurs de $T[4,1]$ et $T[3,2]$.
3. $L[i, j]$ est la somme maximale de la sous pyramide dont le sommet est la case en position (i, j) .
 - a) Que vaut $L[1, j]$?
 - b) Montrer que pour $i > 1$, $L[i, j] = T[i, j] + \max(L[i-1, j], L[i-1, j+1])$.
4.
 - a) Proposer un programme python récursif qui répond au problème.
 - b) Est-ce qu'on effectue plusieurs fois le même calcul ? Est-ce optimal ?
5. Proposer une amélioration en utilisant la programmation dynamique.

● Exercice 13.19

Avec les programmes précédents, on obtient bien la somme maximale, mais pas le chemin qui y mène. Que pourrait-on ajouter pour l'obtenir ?